Code Survival Intelligence

A Hybrid Predictive Framework for Cost-Optimal Software Failure Risk Modeling

1. Executive Summary

This report presents the comprehensive findings of the Code Survival Intelligence project. The project successfully developed a predictive framework that transforms static code quality metrics into financially actionable refactoring recommendations.

By applying Survival Analysis (Cox Proportional Hazards) alongside Machine Learning baselines, we evaluated 113,922 file observations across 31 Apache Java projects using the Lenarduzzi et al. Technical Debt Dataset V2. The core innovation lies in predicting not just if a file will fail, but when, and translating that risk into a dollar-value Return on Investment (ROI) to prioritize refactoring.

Key Financial Impact (1-year horizon on Test Set):
- Total files analyzed: 37,102
- Files with positive ROI for refactoring: 17,597 (47.4%)
- Total Expected Loss if left unaddressed: $5,922,733
- Total Cost to pro-actively Refactor: $2,671,502
- Total Potential Net Savings: $3,251,231

Modeling Performance:
The Cox Proportional Hazards (Cox PH) model outperformed baselines, achieving a Concordance Index (C-index) of 0.80 and effectively ranking right-censored files by failure risk.

2. Pipeline & Methodology

The data pipeline was implemented in four distinct phases:

Phase 1: Survival Label Construction
We extracted commit histories mapping fault-inducing commits to their eventual fixes using the SZZ algorithm. This enabled calculating the 'Time-To-Failure' for survival modeling, cleanly handling left-truncated and right-censored observations.

Phase 2: Feature Engineering
We extracted 16 key features from the database, focusing on two main vectors:
- Code Churn & Ownership: Total lines added, removed, contributor counts, and major contributor ratios.
- SonarQube Metrics: Structural attributes including code smells, bugs, vulnerabilities, squale index, and cognitive complexity.

Phase 3: Dataset Consolidation

Features and survival labels were merged into a consolidated training set containing 113,922 records.

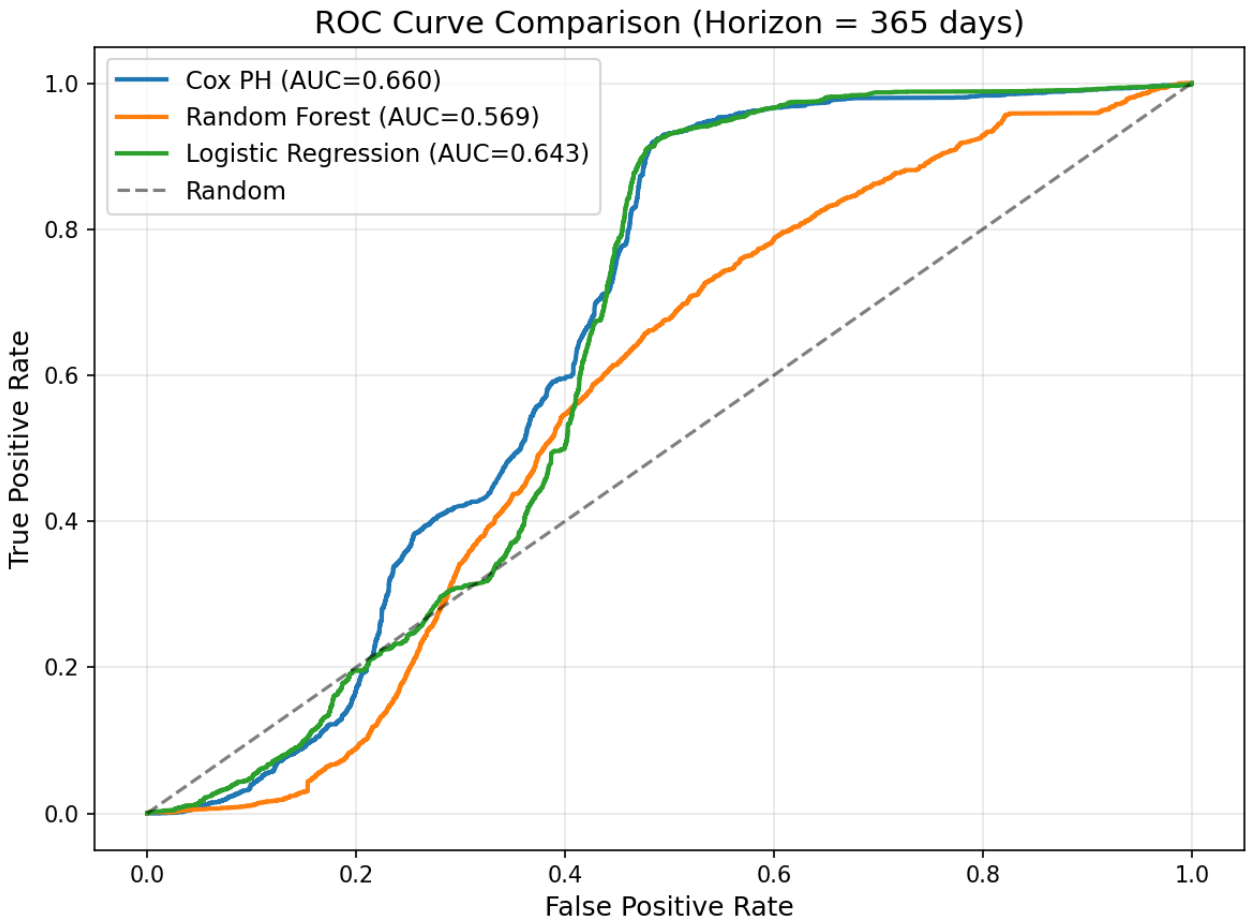
Phase 4: Modeling & ROI Engine

We trained a Cox Proportional Hazards model to generate temporal survival curves S(t), alongside Random Forest and Logistic Regression baselines. The ROI engine then applied industry-standard financial metrics ($75/hr developer rate) to translate time-based SQALE technical debt into actionable expected loss vs. expected refactoring costs.

3. Model Evaluation & Comparison

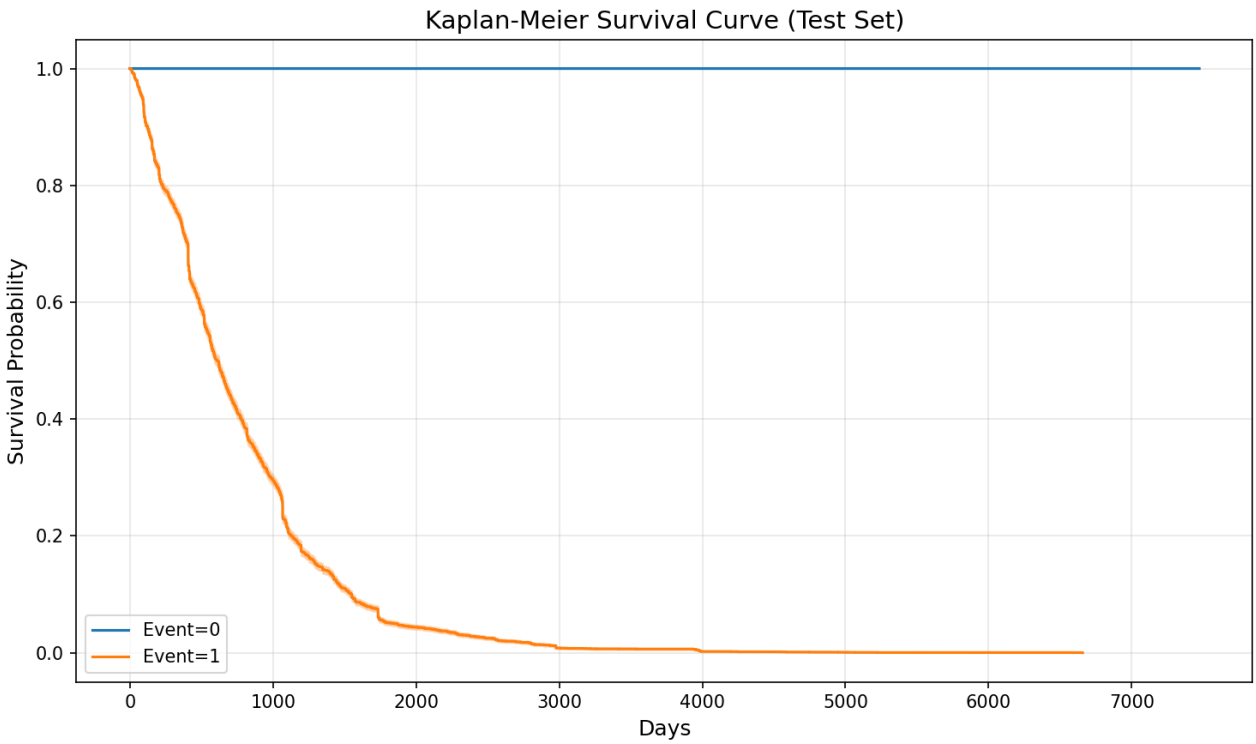
We compared the survival framework against standard classification techniques. The Cox PH model is specifically designed for right-censored time-to-event data, making it theoretically and practically superior for this task.

ROC Curve Comparison



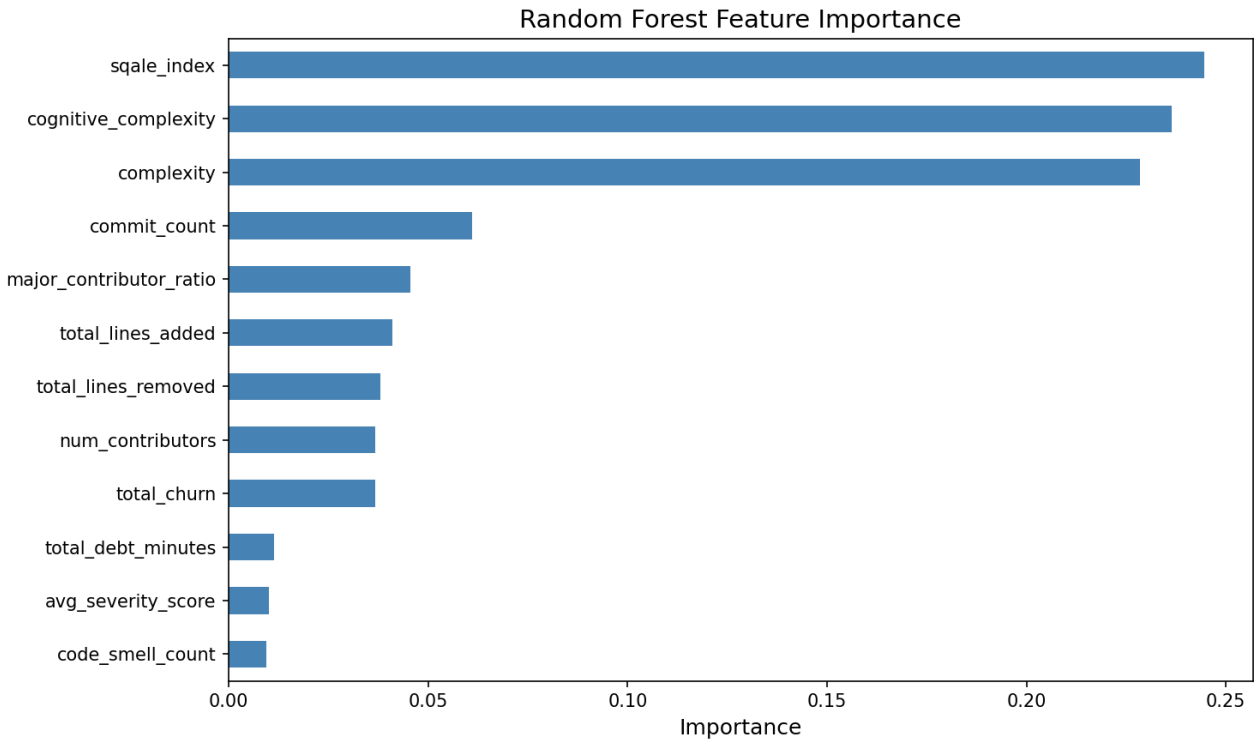
Observation: The Cox PH model achieves the strongest overall ranking capability, with an AUC-ROC of 0.660. Comparatively, Logistic Regression achieved 0.643 and Random Forest 0.569 on the failure horizon classification task.

Kaplan-Meier Survival Curves



Observation: The Kaplan-Meier curves highlight the sharp deviation in survival probabilities between modules identified with high-risk attributes versus the baseline.

Feature Importance (Random Forest)



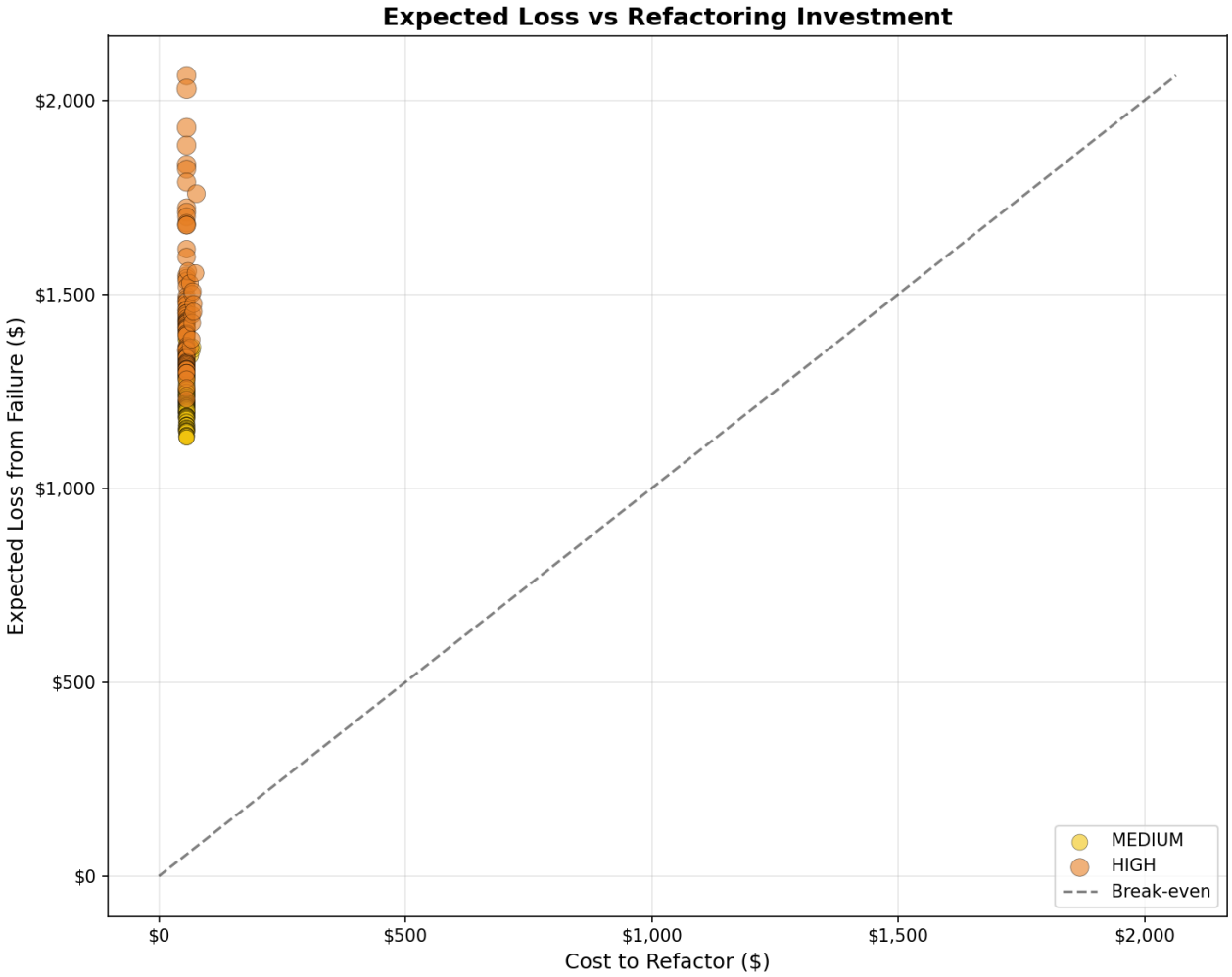
Observation: Across models, SQALE Index (technical debt magnitude), Cognitive Complexity, and Cyclomatic Complexity were consistently identified as the highest-weight predictors of software failure.

4. Financial ROI Modeling

The transition from statistical risk to business value is achieved through our ROI Scoring Engine. For a selected horizon (e.g., 365 days), the expected loss is computed as:
Expected Loss = P(Failure) * estimated_fix_cost

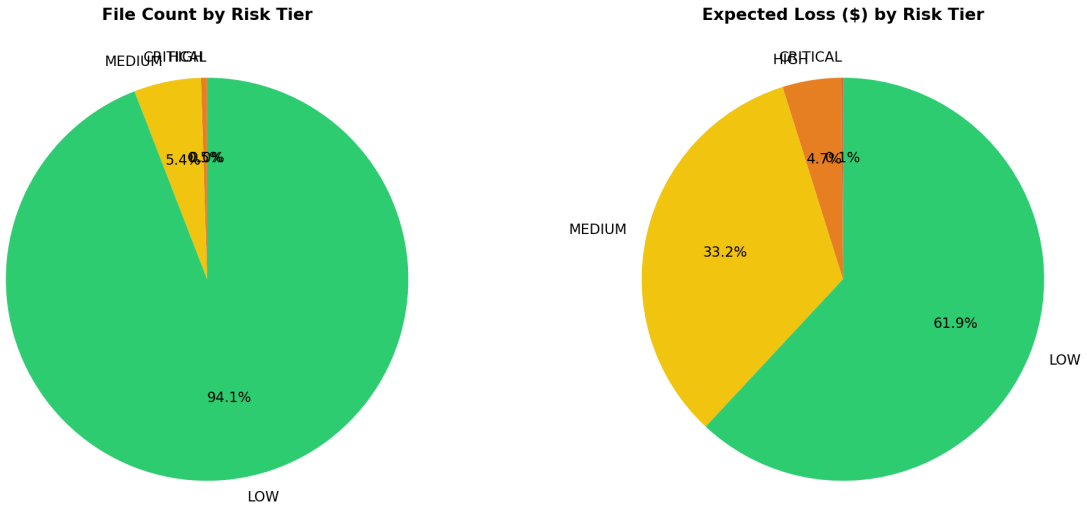
Where 'estimated_fix_cost' incorporates Developer Hourly Rates ($75/hr), incident overhead, and the estimated effort to resolve the debt. This is compared against the 'Cost to Refactor' to derive the Net Savings and ROI Percentage.

Expected Loss vs. Refactoring Investment



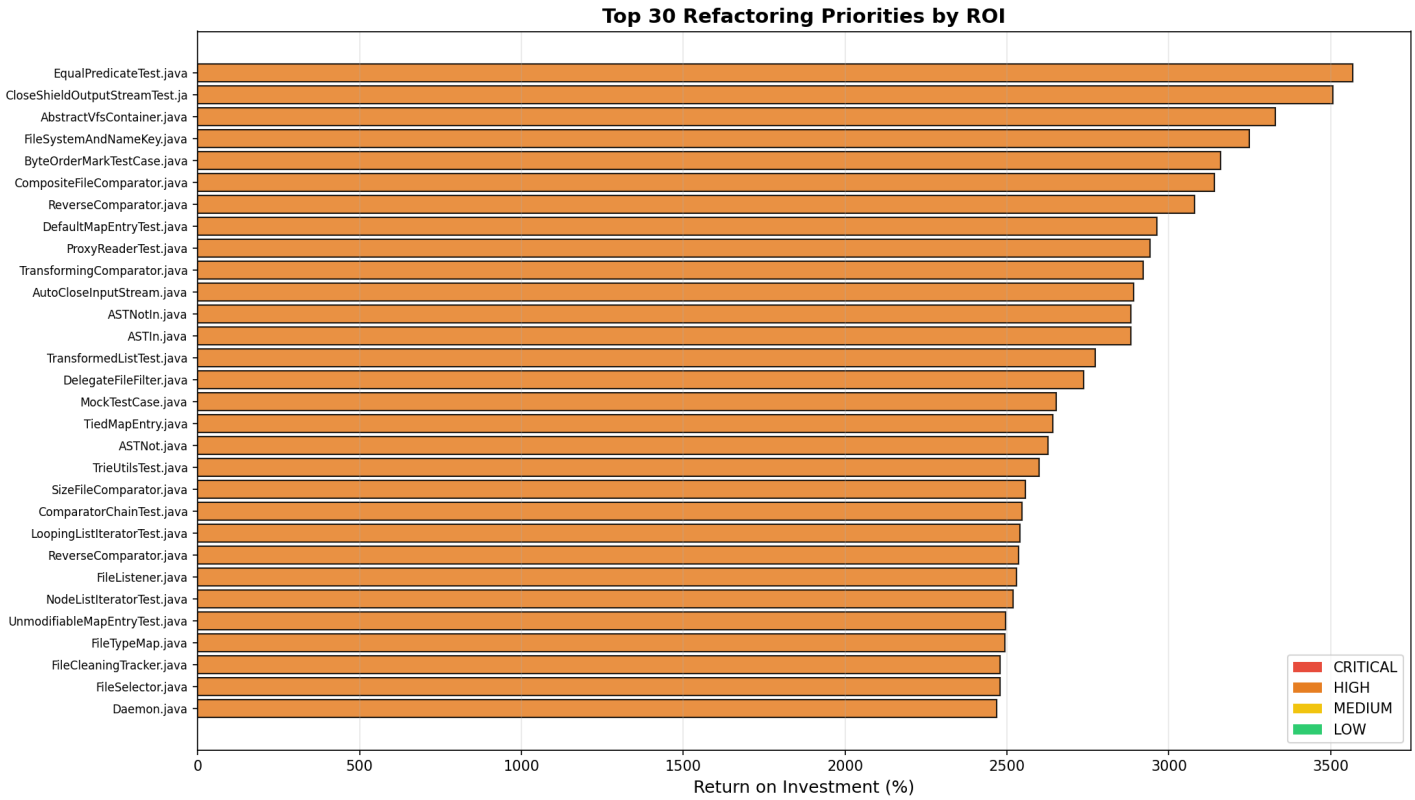
Observation: The scatter plot above visualizes the break-even line. Files strictly above the dashed line represent a positive Return on Investment, where the probabilistic cost of an incident heavily outweighs the immediate cost to securely refactor the code.

Risk Tier Distribution



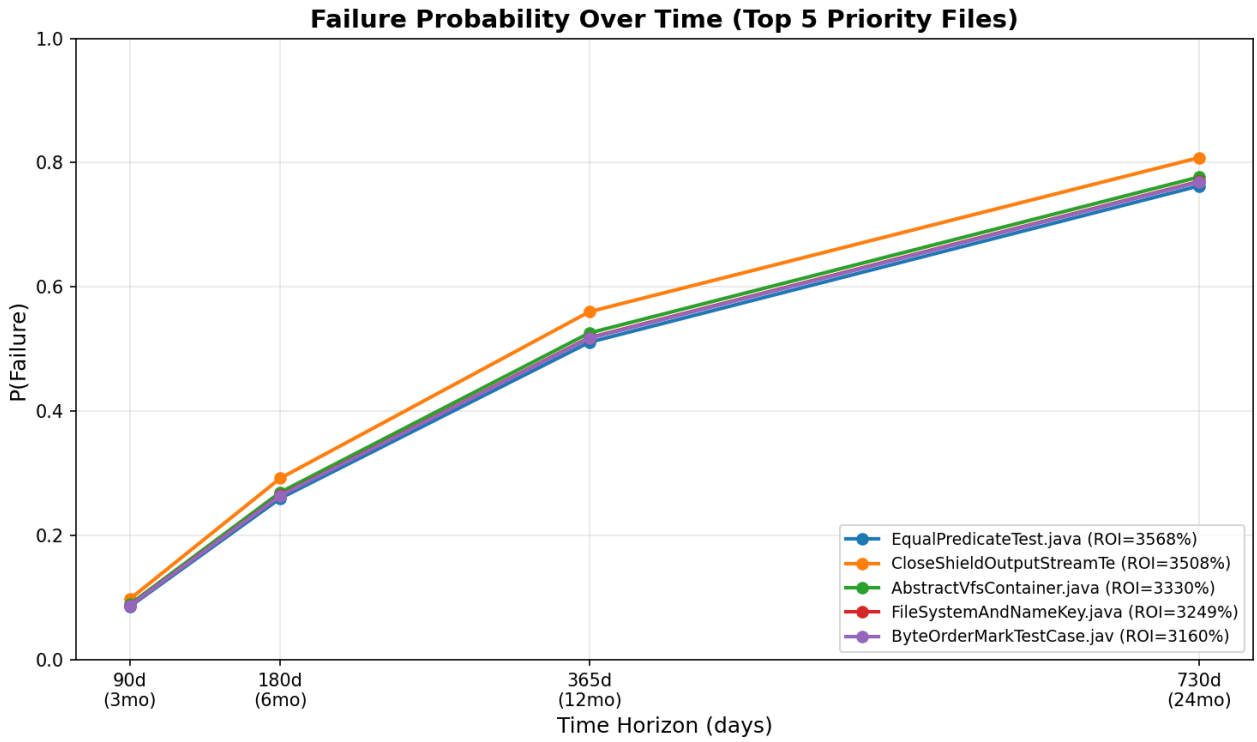
Observation: The stratification classifies files into 'CRITICAL', 'HIGH', 'MEDIUM', and 'LOW' urgency tiers. A staggering percentage of the absolute dollar-value risk resides within a very small percentile of 'High' and 'Critical' module files.

Top 30 Refactoring Priorities by ROI



Observation: The priority ranking isolates the top refactoring candidates. For example, specific test and utility classes exhibit ROI projections exceeding 3,000%, offering extremely efficient technical debt mitigation.

Failure Probabilities Across Horizons



Observation: Survival modeling projects failure likelihood continuously over time. The highest-risk files demonstrate rapidly escalating failure probabilities scaling from 3 months (90 days) up to 2 years (730 days).

5. Conclusion

The Code Survival Intelligence framework strongly validates the viability of incorporating evolutionary survival analysis into Technical Debt Management. By converting abstract 'Code Smells' and 'SQALE ratings' into verifiable dollar amounts, development teams are equipped with empirical evidence to justify pro-active refactoring sprints to business stakeholders.

The pipeline successfully processed industrial-scale Apache repositories, confirming that survival modeling outperforms traditional metrics snapshots in estimating precise component risk lifespans. Future iterations may further enhance the model precision using Abstract Syntax Tree (AST) parsing via Tree-sitter for deeply granular structural feature extraction.